0846. Hand of Straights

Medium | Category: Greedy | LeetCode: https://leetcode.com/problems/hand-of-

Problem

- Can the cards be rearranged into groups of groupSize consecutive values?

Intuition

- The smallest remaining card must start a group (nothing smaller can include it).
- So repeatedly take the smallest card x and consume x, x+1, ..., x+size-1.

Brute Force

- Backtracking over groupings: exponential.

Optimal Approach

- If n % size != 0 return false.
- TreeMap value → count.
- While the map is non-empty: x = firstKey; for v = x..x+size-1: if v is missing return false; decrement (remove at 0).

Worked Example

```
[1,2,3,6,2,3,4,7,8], size 3.

smallest 1 -> take 1,2,3   left {2,3,4,6,7,8}
smallest 2 -> take 2,3,4   left {6,7,8}
smallest 6 -> take 6,7,8   done -> true
```

Java Solution

```java
public class HandOfStraights {
    public boolean isNStraightHand(int[] hand, int groupSize) {
        if (hand.length % groupSize != 0) return false;
        TreeMap<Integer, Integer> count = new TreeMap<>();
        for (int c : hand) count.merge(c, 1, Integer::sum);
        while (!count.isEmpty()) {
            int x = count.firstKey();
            for (int v = x; v < x + groupSize; v++) {
                Integer c = count.get(v);
                if (c == null) return false;
                if (c == 1) count.remove(v);
                else count.put(v, c - 1);
            }
        }
        return true;
    }
}
```

Complexity

- Time: O(n log n). Space: O(n).

Pitfalls

- Same problem as LC 1296 (Divide Array in Sets of K Consecutive Numbers).